

UNIVERSITY OF SCIENCE
VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY



Research Proposal

Dashcam Video Question Answering

CS412 – Computer Vision

Team Member

22125056 – Le Phat Minh
22125078 – Dinh Cao Minh Quan
22125085 – O Hon Sam

Theoretical Teacher : Tran Thai Son

Laboratory Teacher : Pham Minh Hoang
Vo Hoai Viet

Ho Chi Minh City, November 12, 2025

Contents

1	User Guide	3
2	Harris Corner Detector	3
2.1	Algorithm Description	3
2.2	Implementation Details	4
2.3	Evaluation	4
2.3.1	Strengths	4
2.3.2	Weaknesses	7
3	DoG Detector	8
3.1	Algorithm Description	8
3.2	Implementation Details	9
3.3	Evaluation	9
3.3.1	Strengths	9
3.3.2	Weaknesses	10
4	Blob Detector	10
4.1	Algorithm Description	11
4.2	Implementation Details	11
4.3	Evaluation	11
4.3.1	Strengths	11
4.3.2	Weaknesses	12
5	SIFT Descriptor	13
5.1	Algorithm Description	13
5.2	Implementation Details	13
5.3	Evaluation	13
5.3.1	Strengths	13
5.3.2	Weaknesses	14
6	LBP Descriptor	14
6.1	Algorithm Description	14
6.2	Implementation Details	15

6.3	Evaluation	15
6.3.1	Strengths	15
6.3.2	Weaknesses	15
7	Feature Matching	15
7.1	Intuition	16
7.2	Matching Pipeline	16
7.3	Implementation Details	16
7.4	Comparison: SIFT vs LBP Matchers	17
7.5	Visual Results	17
7.6	Effect of Detector + Descriptor Pairing	20
7.7	Summary	20
8	Experiment	21
8.1	Experimental Setup	21
8.1.1	Dataset	21
8.1.2	Parameter Settings	21
8.2	Evaluation Methodology	21
8.3	Results	21

1 User Guide

The lab was conducted on **Ubuntu 24.04** with OpenCV 4.10.0 and GCC 13.3.0. Compilation was managed using a Makefile.

Please compile the program using **make** before running the following commands:

Usage	
<code>program.exe <command> [arguments]</code>	Run the program with a specified cmd, and args.

Available Commands	
<code>harris <image.jpg></code>	Detect keypoints using Harris Corner detector.
<code>blob <image.jpg></code>	Detect keypoints using Blob detector.
<code>dog <image.jpg></code>	Detect keypoints using DoG (SIFT).
<code>m harris sift <img1> <img2></code>	Match images using Harris + SIFT.
<code>m dog sift <img1> <img2></code>	Match images using DoG + SIFT.
<code>m blob sift <img1> <img2></code>	Match images using Blob + SIFT.
<code>m harris lbp <img1> <img2></code>	Match images using Harris + LBP.
<code>m dog lbp <img1> <img2></code>	Match images using DoG + LBP.
<code>m blob lbp <img1> <img2></code>	Match images using Blob + LBP.
<code>h</code>	Display help message.

Keyboard Controls	
<code>esc</code>	Quit video window.

2 Harris Corner Detector

Harris Corner Detector is a classical algorithm used to identify corners (interest points) in an image. Corners are regions where the image intensity changes significantly in multiple directions, making them robust and repeatable features for matching and recognition tasks.

2.1 Algorithm Description

The algorithm works by computing the Harris response matrix for each pixel in the image, which captures the intensity gradients in both the x and y directions. A corner response is then calculated based on the eigenvalues of this matrix, and non-maximum suppression is applied to identify the strongest corners.

- **Input:** Grayscale image.
- **Output:** List of detected corner points (keypoints).
- **Parameters:**
 - **blockSize:** Size of the neighborhood considered for corner detection. (default: 2)
 - **ksize:** Aperture parameter for the Sobel operator used to compute image gradients. (default: 3)
 - **k-x100:** Harris detector free parameter scaled by 100, typically in the range [4, 6]. (default: 4)
 - **threshold:** Minimum response value to consider a point as a corner. (default: 100)

2.2 Implementation Details

1. **Gradient Computation:** Compute the image gradients in the x and y directions using the Sobel operator
2. **Harris Matrix Calculation:** For each pixel, compute the Harris matrix by the products of gradients then applying a Gaussian filter to obtain weighted sum within the neighborhood defined by **blockSize**.
3. **Corner Response Calculation:** Calculate the corner response using the determinant and trace of the Harris matrix.

$$R = \det(M) - k \cdot (\text{trace}(M))^2 \quad (1)$$

where M is the Harris matrix at each pixel, and k is the sensitivity factor (typically between 0.04 and 0.06).

4. **Thresholding and Non-Maximum Suppression:** Identify potential corners by applying the threshold to the corner response values. Then, perform non-maximum suppression to retain only the strongest corners in the local neighborhood, and to avoid multiple detections of the same corner.
5. **Keypoint Creation:** Convert the detected corner points into keypoints format for further processing or visualization.

2.3 Evaluation

2.3.1 Strengths

- **Robustness to Noise:** Harris Corner Detector is relatively robust to noise due to the use of gradient information.

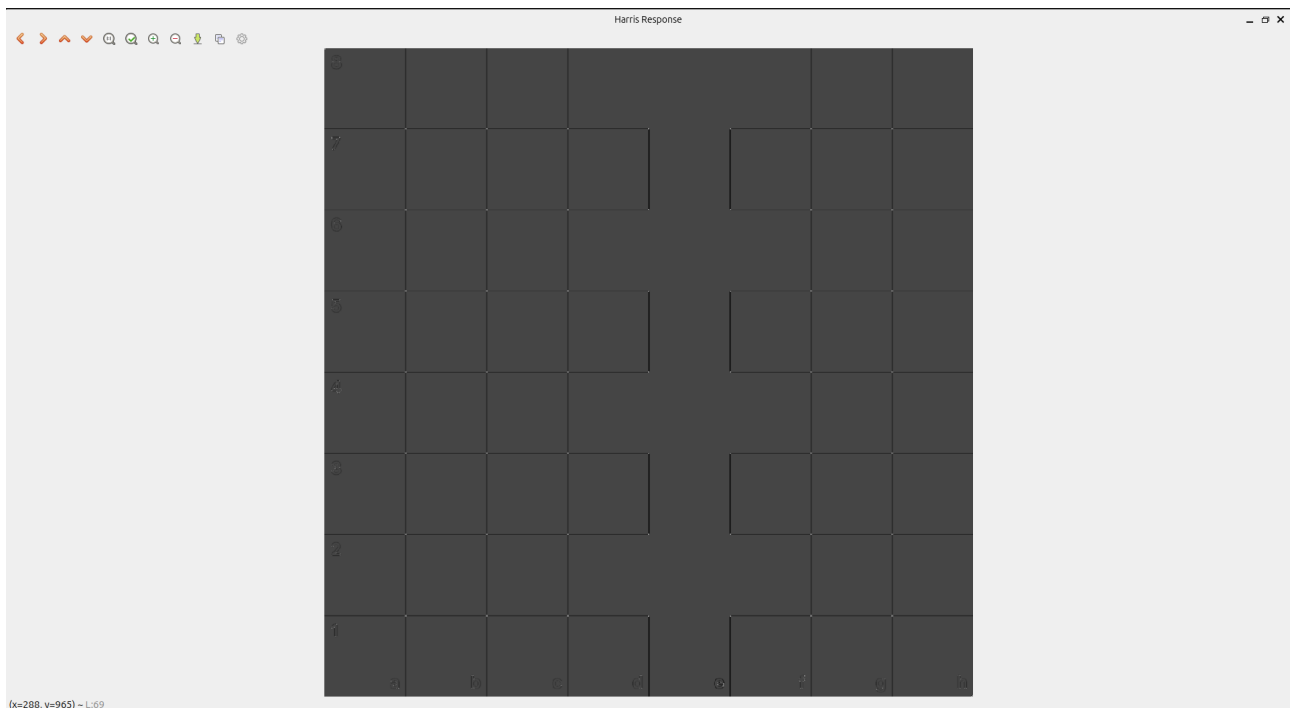


Figure 1: Harris Response Heatmap.

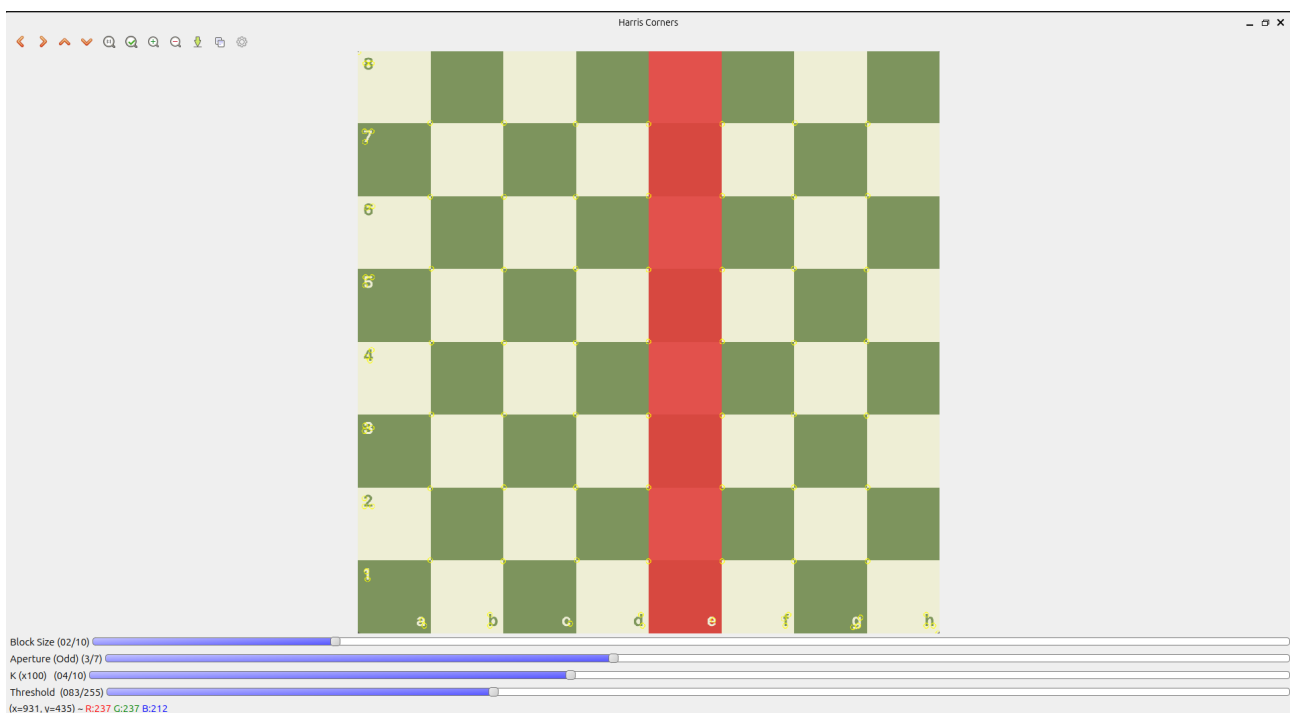
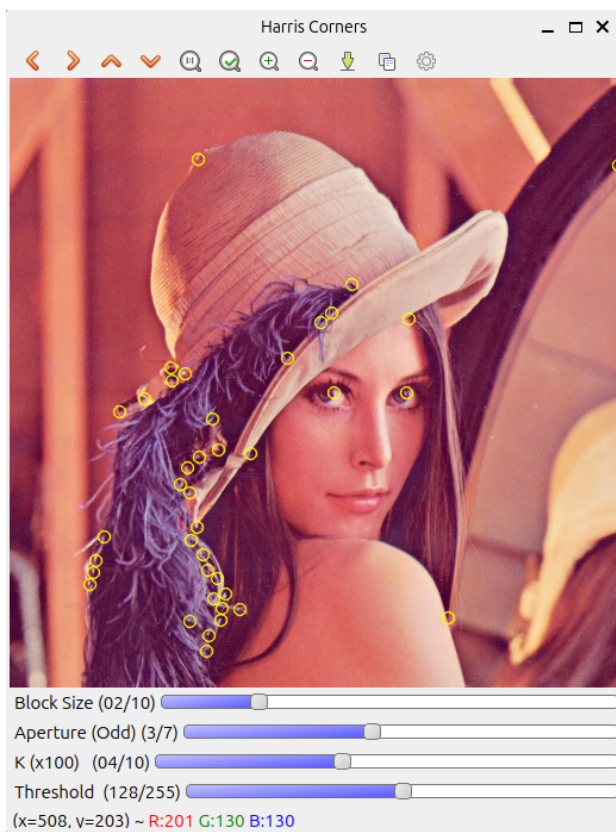
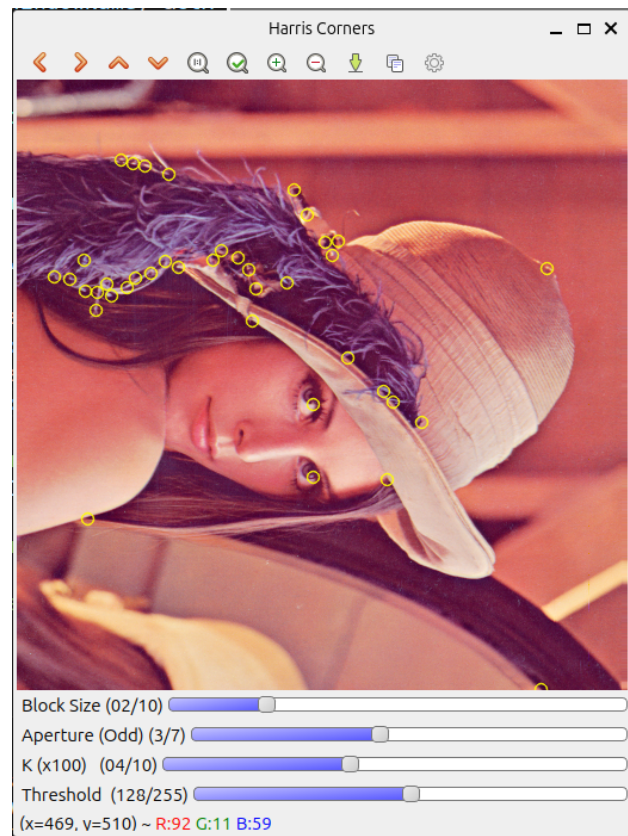


Figure 2: Harris Corners Detected.

- **Rotation Invariance:** The detector is invariant to image rotation, making it suitable for various applications.



(a) Normal Image.



(b) Rotated Image.

Figure 3: Illustration of Harris Corner Detection for Normal and Rotated Images.

- **Computational Efficiency:** The algorithm is computationally efficient and can be implemented in real-time applications.

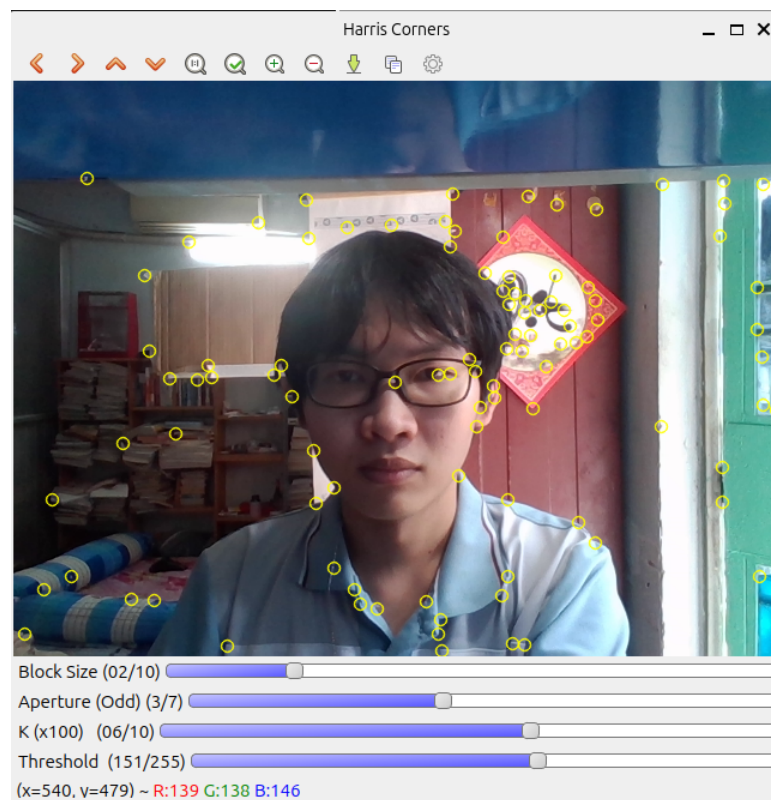


Figure 4: Harris Corners Detection on Continuous Images Captured by Camera.

2.3.2 Weaknesses

- **Scale Sensitivity:** The Harris Corner Detector is not inherently scale-invariant, which can lead to missed detections at different scales.

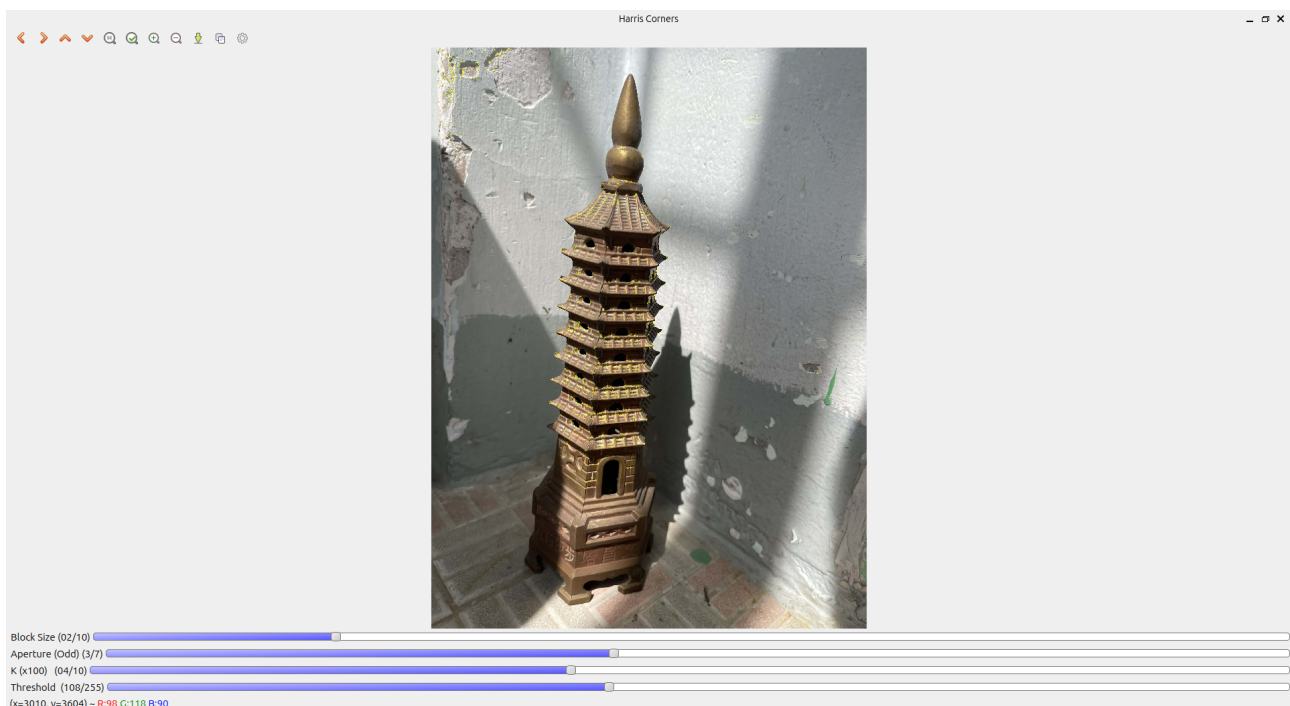


Figure 5: Normal Image.

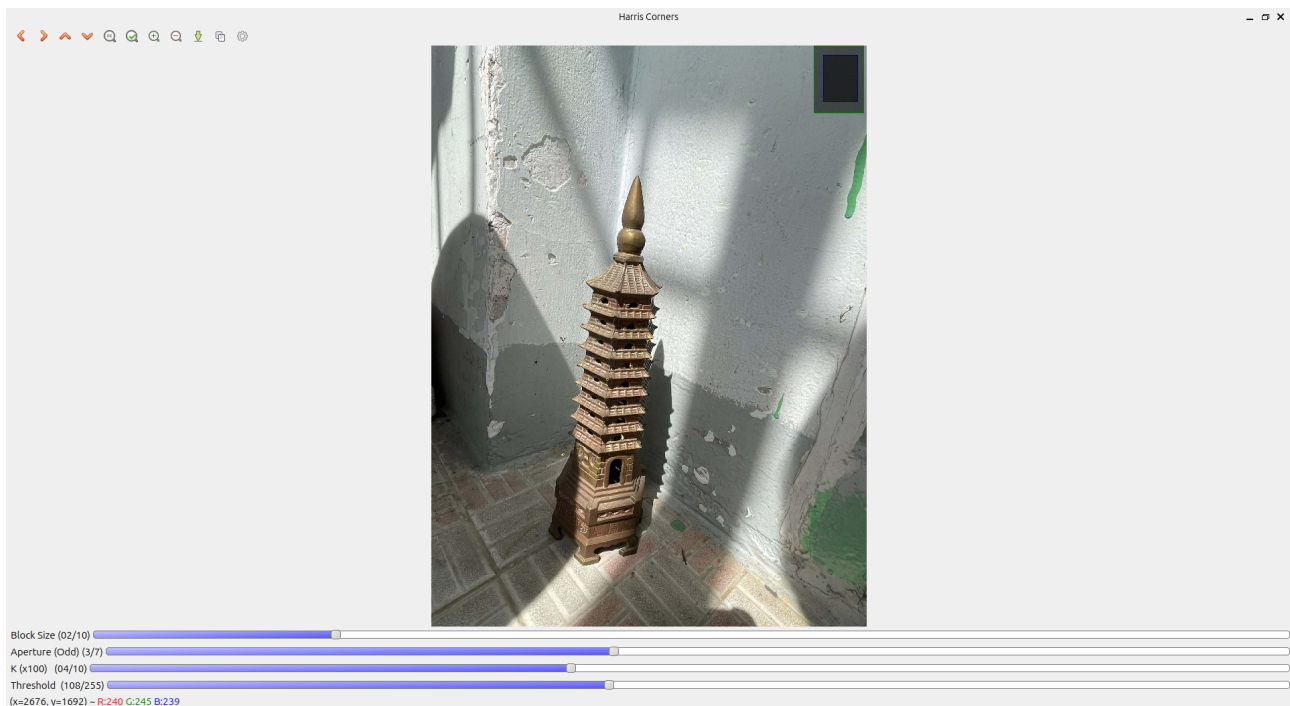


Figure 6: Scaled Image.

- **Parameter Sensitivity:** The performance of the detector can be sensitive to the choice of parameters, requiring careful tuning for different images.
- **Limited to Corners:** The detector is specifically designed to detect corners and may not perform well in regions with low texture or flat areas.
- **False Positives:** In some cases, the detector may produce false positives, especially in noisy images or images with repetitive patterns.
- **Not Fully Affine Invariant:** The Harris Corner Detector is not invariant to scaling, sheering transformations, which can limit its effectiveness in scenarios involving significant viewpoint changes.

3 DoG Detector

The Difference of Gaussians (DoG) detector is a popular algorithm for detecting blob-like structures and keypoints in images. The DoG is an efficient approximation to the Laplacian of Gaussian (LoG), the canonical operator for blob detection in scale-space theory. It is widely used as the basis for the SIFT (Scale-Invariant Feature Transform) detector due to its robustness to changes in scale and moderate changes in illumination.

3.1 Algorithm Description

The DoG detector works by subtracting two blurred versions of the same image, each with a different Gaussian kernel (i.e., different standard deviations). This process highlights regions in the image where the intensity changes rapidly, which often correspond to keypoints or blobs.

- **Input:** Grayscale image.
- **Output:** List of detected keypoints (blobs).
- **Parameters:**
 - **sigma1-x10:** Standard deviation for the first Gaussian blur, scaled by 10. (default: 1)
 - **sigmaDiff:** Difference in standard deviation between the two Gaussian blurs, scaled by 10. (default: 5)
 - **kernelSize:** Size of the Gaussian kernel. (default: 5)
 - **threshold:** Minimum response value to consider a point as a keypoint. (default: 100)

3.2 Implementation Details

1. **Gaussian Blurring:** Apply two Gaussian blurs to the input image with different standard deviations (**sigma1** and **sigma1 + sigmaDiff**).
2. **Difference Calculation:** Subtract the two blurred images to obtain the DoG response, which enhances blob-like structures.
3. **Normalization:** Normalize the DoG response to a standard range for thresholding. This is an optional but practical step for consistent thresholding.
4. **Thresholding and Non-Maximum Suppression:** Identify potential keypoints by applying a threshold to the DoG response. Then, perform non-maximum suppression to retain only the strongest keypoints in the local neighborhood.
5. **Keypoint Creation:** Convert the detected points into keypoints format for further processing or visualization.

3.3 Evaluation

3.3.1 Strengths

- **Scale Robustness:** In many cases, the DoG detector is more robust to changes in scale compared to corner detectors like Harris, making it suitable for detecting features at multiple sizes. Take a look at Figure 7.
- **Blob Detection:** Effectively detects blob-like structures and regions with significant intensity variation.
- **Good for SIFT:** Since it is scale-robust, the algorithm can be utilized to form the basis for the SIFT detector, which is widely used for robust feature matching.



(a) Normal Image.



(b) Scaled Image.

Figure 7: Illustration of DoG Detection for Normal and Scaled Images.

3.3.2 Weaknesses

- **Parameter Sensitivity:** The performance depends on the choice of `sigma1`, `sigmaDiff`, and `threshold`, which may require tuning for different images.
- **Not Fully Affine Invariant:** While more robust to scale, DoG is not invariant to affine transformations such as shearing or extreme perspective changes.
- **May Miss Corners:** DoG is optimized for blobs and may not detect sharp corners as reliably as Harris.
- **False Positives:** In noisy images, DoG may detect spurious blobs unless parameters are carefully chosen.

4 Blob Detector

A blob is a region of the image that is brighter or darker than its surroundings in a compact, roughly roundish way. Mathematically, a blob is where the second derivative operator detects a maximum or minimum in intensity when viewed at a certain scale.

4.1 Algorithm Description

The Blob Detector identifies such regions by thresholding the grayscale image at multiple levels and searching for connected regions (blobs) that satisfy certain geometric and intensity criteria. The algorithm can be implemented using OpenCV's `SimpleBlobDetector` or a custom approach.

- **Input:** Grayscale image.
- **Output:** List of detected keypoints (blobs).
- **Parameters:**
 - `minThreshold`, `maxThreshold`: Range of thresholds for binarization. (default: 50, 255)
 - `thresholdStep`: Step size for threshold increments. (default: 5)
 - `minArea`, `maxArea`: Minimum and maximum area for detected blobs. (default: 50, 50000)
 - `minCircularity`: Minimum circularity required for a blob. (default: 40)
 - `minConvexity`: Minimum convexity required for a blob. (default: 40)
 - `minInertia`: Minimum inertia ratio required for a blob. (default: 40)
 - `minDistBetweenBlobs`: Minimum distance between blob centers to be considered separate. (default: 10.0f)

4.2 Implementation Details

1. **Multi-level Thresholding:** The image is thresholded at multiple levels between `minThreshold` and `maxThreshold` using a specified `thresholdStep`.
2. **Contour Detection:** For each thresholded image, contours are found to identify candidate blobs.
3. **Blob Filtering:** Each candidate blob is filtered based on area, circularity, convexity, and inertia ratio.
4. **Grouping and Keypoint Creation:** Centers of blobs detected at different thresholds are grouped if they are close together (within `minDistBetweenBlobs`), and the average center and size are used to create keypoints.
5. **Visualization:** The detected blobs are visualized as keypoints, often drawn as circles on the original image.

4.3 Evaluation

4.3.1 Strengths

- **Robust to Scale:** By thresholding at multiple levels and grouping detections, the blob detector can find blobs of various sizes.

- **Shape Filtering:** The use of circularity, convexity, and inertia filters allows for selective detection of blobs with specific shapes.
- **Useful for Counting and Tracking:** Well-suited for applications where objects are approximately round or elliptical, such as coins, cells, or balls.

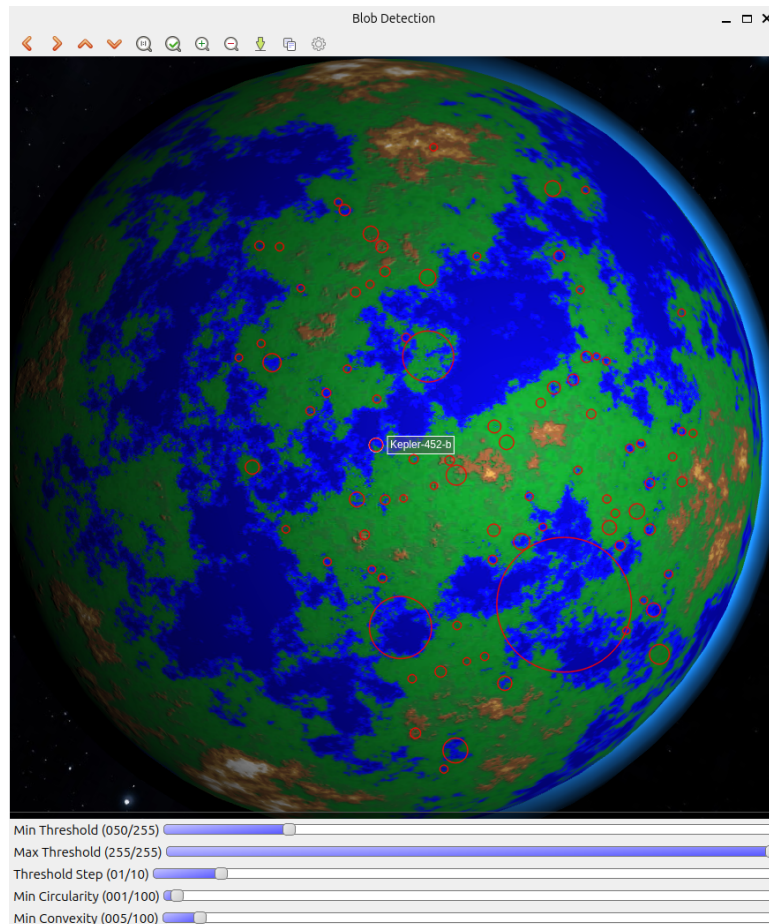


Figure 8: Blob Detector identifying regions in Kepler planet.

4.3.2 Weaknesses

- **Parameter Sensitivity:** The detector's performance is highly dependent on parameter settings, which may require manual tuning for different images.
- **Limited to Blob-like Structures:** Not suitable for detecting corners or elongated features.
- **False Positives in Noisy Images:** May detect spurious blobs in the presence of noise or cluttered backgrounds.
- **Not Affine Invariant:** The detector is not robust to affine transformations such as shearing or extreme perspective changes.
- **Overlapping Blobs:** May have difficulty distinguishing between closely spaced or overlapping blobs.

5 SIFT Descriptor

The Scale-Invariant Feature Transform (SIFT) is a widely used algorithm for extracting distinctive and robust feature descriptors from images. SIFT descriptors are designed to be invariant to image scale and rotation, and partially invariant to changes in illumination and affine transformations. This makes them highly effective for matching features across different views of an object or scene.

5.1 Algorithm Description

SIFT operates in two main stages: keypoint detection (often using the DoG detector) and descriptor computation. For each detected keypoint, SIFT computes a descriptor vector that characterizes the local image structure.

- **Input:** Grayscale image and a set of keypoints.
- **Output:** A set of 128-dimensional feature descriptors, one for each keypoint.
- **Parameters:**
 - **nOctaveLayers:** Number of layers in each octave (affects scale-space sampling). (default: 3)
 - **contrastThreshold:** Threshold to filter out weak features. (default: 4)
 - **edgeThreshold:** Threshold to filter out edge-like features. (default: 100)
 - **sigma:** Initial Gaussian blur applied to the image, scaled by 10. (default: 16)

5.2 Implementation Details

1. **Keypoint Detection:** Keypoints are detected using the Difference of Gaussians (DoG) method, which identifies scale-space extrema.
2. **Orientation Assignment:** For each keypoint, the dominant orientation is determined based on local image gradients, ensuring rotation invariance.
3. **Descriptor Computation:** A 16x16 neighborhood around each keypoint is divided into 4x4 subregions. For each subregion, an 8-bin histogram of gradient orientations is computed, resulting in a 128-dimensional descriptor vector.
4. **Normalization:** The descriptor vector is normalized to enhance robustness to illumination changes.

5.3 Evaluation

5.3.1 Strengths

- **Scale and Rotation Invariance:** SIFT descriptors are robust to changes in scale and rotation, making them suitable for matching features across images with different viewpoints.

- **Distinctiveness:** The 128-dimensional descriptor captures rich local image structure, enabling reliable matching even in cluttered scenes.
- **Robustness to Illumination:** Normalization and gradient-based representation provide partial invariance to changes in lighting.

5.3.2 Weaknesses

- **Computational Cost:** SIFT is more computationally intensive than simpler descriptors like LBP, which may limit its use in real-time applications.
- **Not Fully Affine Invariant:** While robust to scale and rotation, SIFT is only partially invariant to affine transformations such as shearing.
- **High Dimensionality:** The 128-dimensional descriptor increases memory usage and matching time compared to more compact descriptors.

6 LBP Descriptor

The Local Binary Pattern (LBP) descriptor is a simple yet powerful texture operator that labels the pixels of an image by thresholding the neighborhood of each pixel and considering the result as a binary number. LBP is widely used for texture classification, face recognition, and as a feature descriptor for keypoints in computer vision tasks.

6.1 Algorithm Description

LBP works by comparing each pixel with its surrounding neighbors in a local window (typically 3×3). For each neighbor, a binary value is assigned: 1 if the neighbor's intensity is greater than or equal to the center pixel, 0 otherwise. The resulting binary pattern is then converted to a decimal value, which represents the local texture.

- **Input:** Grayscale image and a set of keypoints.
- **Output:** For each keypoint, a histogram-based descriptor representing the local texture.
- **Parameters:**
 - **Radius (radius):** Determines the distance from the center pixel to its neighbors when computing the LBP code. A small radius (e.g., 1) captures fine local texture, while a larger radius allows the descriptor to capture coarser patterns. However, increasing the radius may also make the descriptor more sensitive to noise and less discriminative for small features. (default: 1)
 - **Patch Size (patchSize):** Specifies the size of the region around each keypoint used to compute the LBP histogram. A small patch focuses on very local texture, while a larger patch includes more context but may dilute the descriptor's sensitivity to fine details. If the patch is too large, it may also include irrelevant background information. (default: 16)

6.2 Implementation Details

1. **Neighborhood Sampling:** For each keypoint, extract a local patch around the keypoint location.
2. **LBP Code Calculation:** For each pixel in the patch, compute the LBP code by thresholding the neighborhood and converting the binary result to a decimal value.
3. **Histogram Construction:** Build a histogram of LBP codes within the patch to form the descriptor for the keypoint.
4. **Normalization:** Normalize the histogram to make the descriptor robust to illumination changes.

6.3 Evaluation

6.3.1 Strengths

- **Computational Efficiency:** LBP is very fast to compute and suitable for real-time applications.
- **Texture Sensitivity:** Captures fine texture details, making it effective for texture and pattern recognition.
- **Illumination Robustness:** The descriptor is invariant to monotonic gray-level changes, providing robustness to lighting variations.
- **Compact Representation:** The histogram-based descriptor is compact and easy to compare.

6.3.2 Weaknesses

- **Not Rotation Invariant:** Standard LBP is sensitive to rotation; rotated patterns may yield different codes.
- **Limited Discriminative Power:** LBP may not distinguish well between complex patterns or in highly textured scenes.
- **Sensitive to Noise:** Small intensity changes due to noise can affect the LBP code.
- **Not Scale Invariant:** LBP is not robust to changes in scale; the descriptor is fixed to the chosen neighborhood size.

7 Feature Matching

Feature matching is the process of identifying corresponding keypoints between two images. Once keypoints have been detected (e.g., Harris, Blob, or DoG) and their local neighborhoods have been encoded using a descriptor (e.g., SIFT or LBP), a matching algorithm determines which features from one image correspond to those in another.

At a high level, matching works by comparing descriptor vectors. If the descriptors are similar - that is, they represent similar local image structure - the points are considered potential matches.

7.1 Intuition

Matching becomes a nearest-neighbor search in descriptor space:

- **Keypoint detector:** decides *where* to look.
- **Descriptor:** decides *how* the neighborhood is represented.
- **Matcher:** decides *which points correspond*.

A matching pipeline requires all three components to work well together.

7.2 Matching Pipeline

1. **Keypoint Detection** Detect interest points using Harris, Blob, or DoG.
2. **Descriptor Extraction** For each keypoint, compute its descriptor:
 - **SIFT:** 128-dimensional gradient histogram (robust, distinctive).
 - **LBP:** Binary texture pattern (compact, less distinctive).
3. **Feature Matching** Use a distance function between descriptors:
 - **L2 distance:** for SIFT.
 - **Hamming distance:** for LBP.
4. **Lowe's Ratio Test (for SIFT)** Prevents ambiguous matches:

$$\frac{\text{distance to best match}}{\text{distance to 2nd best match}} < \tau$$

where τ is typically 0.7–0.8. (default: 0.75)

7.3 Implementation Details

In our project, feature matching is implemented using OpenCV's `BFMatcher`:

- For SIFT:
 - `cv::NORM_L2`
 - Use KNN matching ($k = 2$) + ratio test.
- For LBP:
 - `cv::NORM_Hamming`
 - Use KNN matching ($k = 2$) + ratio test.

Visualization is done using `cv::drawMatches`, highlighting the connection between corresponding keypoints.

7.4 Comparison: SIFT vs LBP Matchers

Aspect	SIFT Matching	LBP Matching
Distinctiveness	High (rich gradient info)	Moderate (binary texture only)
Invariance	Strong to scale, rotation, lighting	Limited (not scale-invariant)
Speed	Slower	Faster
Robustness	Excellent in cluttered scenes	Sensitive to noise, illumination
Typical Use	Object matching, recognition	Texture classification

7.5 Visual Results

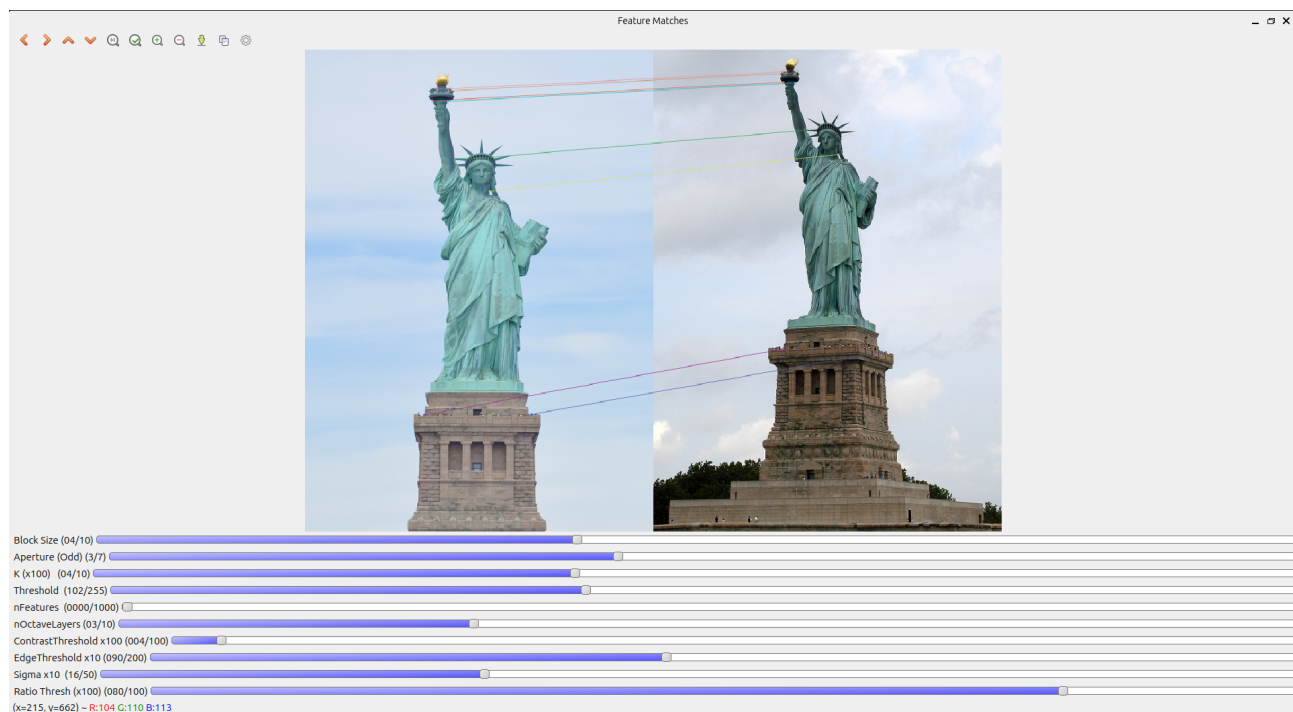


Figure 9: Matching results using Harris + SIFT for Statue of Liberty.

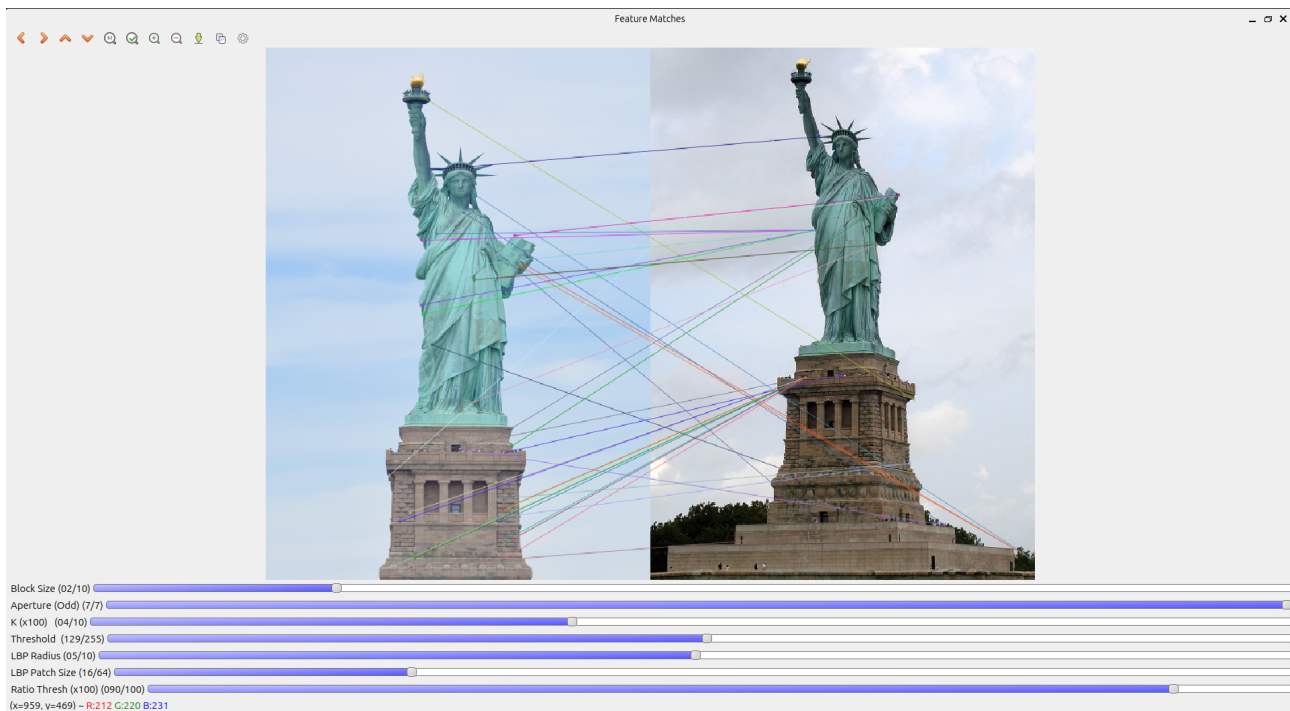


Figure 10: Matching results using Harris + LBP for Statue of Liberty.

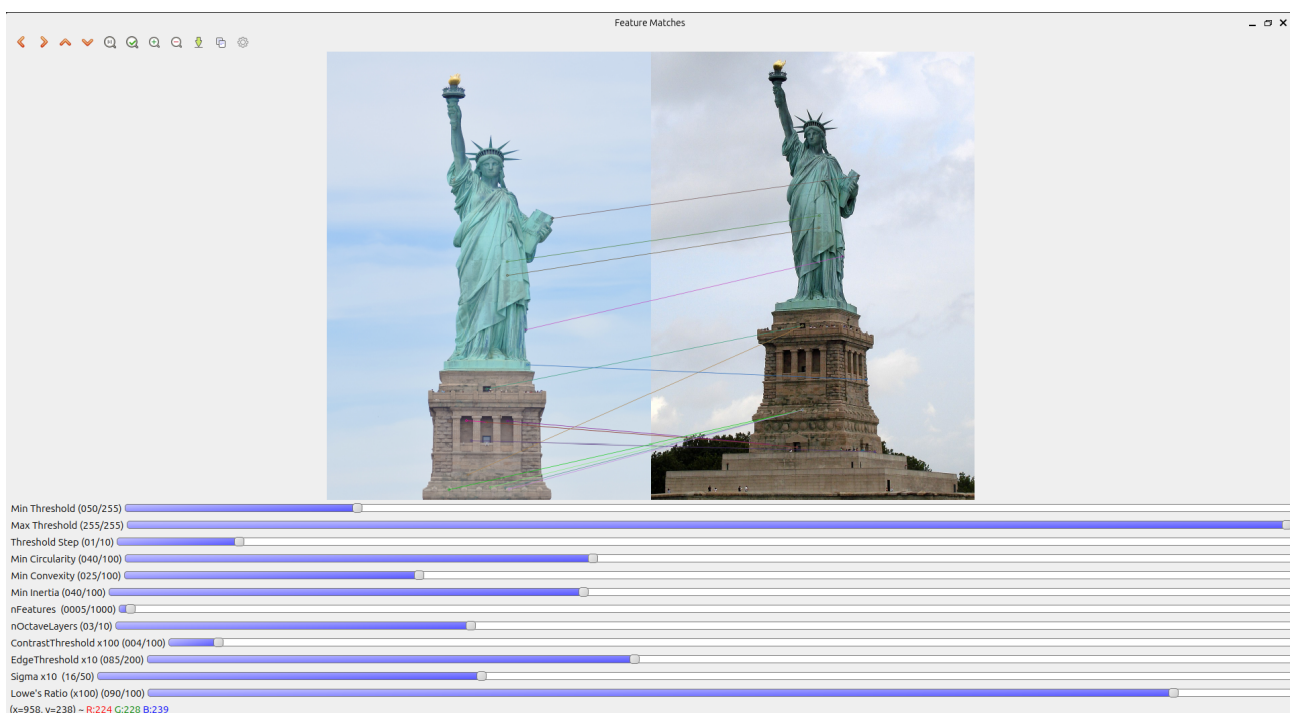


Figure 11: Matching results using Blob + SIFT for Statue of Liberty.



Figure 12: Matching results using Blob + LBP for Statue of Liberty.



Figure 13: Matching results using DoG + SIFT for Statue of Liberty.

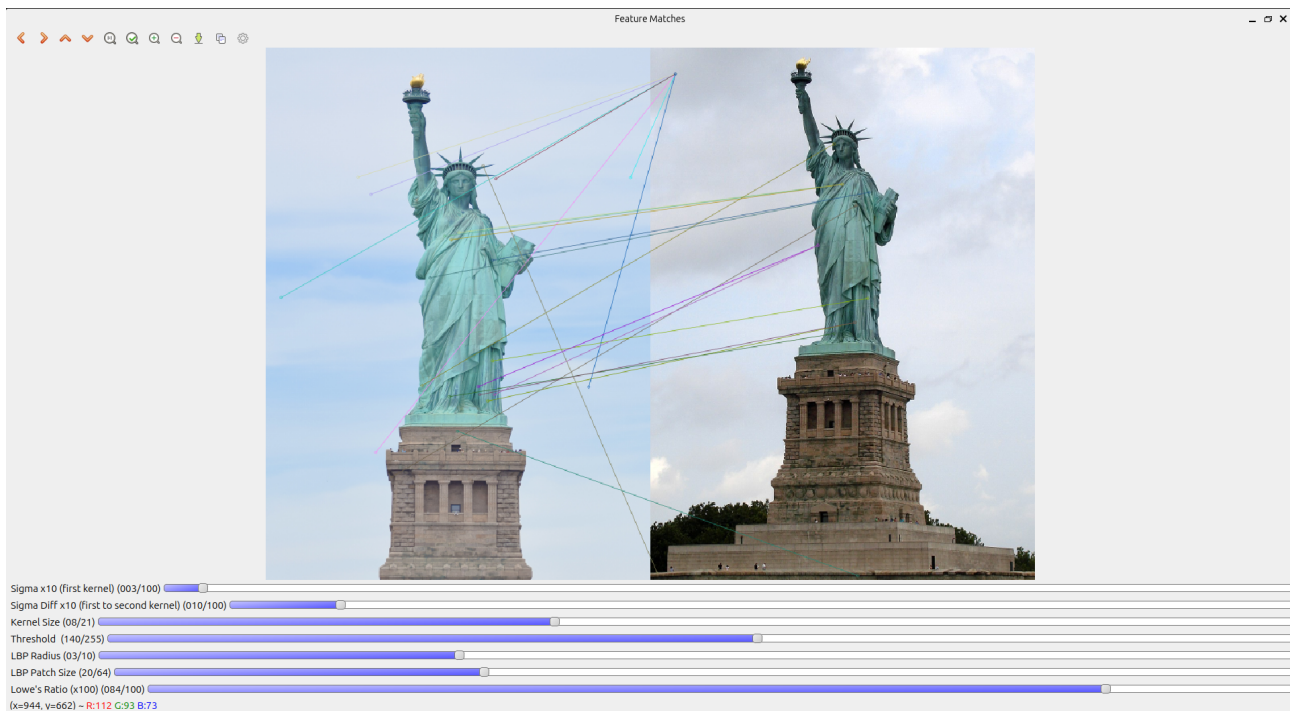


Figure 14: Matching results using DoG + LBP for Statue of Liberty.

7.6 Effect of Detector + Descriptor Pairing

- **Harris + SIFT**: Good for corners; lacks scale invariance, so fewer matches across large scale changes.
- **Blob + SIFT**: Works well when objects contain blob-like or rounded structures.
- **DoG + SIFT**: The most stable and widely used combination. Features are both detected and described in scale-space, which yields excellent matching stability.
- **Harris/Blob/DoG + LBP**: Efficient but weaker matching; good if speed is more important than accuracy.

7.7 Summary

Feature matching succeeds when:

- The detector finds stable, repeatable keypoints.
- The descriptor encodes enough local structure.
- The matching algorithm removes ambiguous and false matches.

Among all tested combinations, **DoG + SIFT** yields the best performance due to its scale-space foundation and robust descriptors.

8 Experiment

8.1 Experimental Setup

8.1.1 Dataset

The evaluation was carried out according to the project specifications. A dataset consisting of five distinct objects was constructed, with each object photographed in three different conditions involving variations in viewpoint and illumination. All experiments were performed using both the static image mode the implemented system.

8.1.2 Parameter Settings

While the system includes OpenCV trackbars for interactive parameter tuning, all experiments in this report were carried out using the default parameter configurations provided in the implementation. Using fixed settings ensures that every detector and descriptor is evaluated under identical conditions, enabling a fair and consistent comparison across all test cases.

8.2 Evaluation Methodology

A three-part evaluation strategy was used to assess the performance of the system:

1. **Keypoint detection quality:** Each detector was evaluated based on the number of detected keypoints and their spatial distribution within the image.
2. **Descriptor and matching effectiveness:** Using both SIFT and LBP descriptors, we measured the number of valid correspondences obtained for each pair of images.
3. **Computational efficiency:** The average processing time was recorded for each configuration using `cv::getTickCount()`.

For every object in the dataset, Image 1 was matched with Image 2 and Image 3.

8.3 Results

Detector	Avg. Keypoints	Min	Max
Harris	267.7	7	752
Blob	62.5	1	245
DoG	241.1	29	749

Table 1: Keypoints Statistics for each detector algorithm.

	Harris + SIFT	Harris + LBP	Blob + SIFT	Blob + LBP	DoG + SIFT	DoG + LBP
kepler1 vs kepler2	7	4	9	1	155	17
kepler2 vs kepler3	4	2	7	1	149	16
Lenna1 vs Lenna2	3	4	0	0	1	3
Lenna1 vs Lenna3	0	2	0	11	11	9
Statue1 vs Statue2	4	1	1	0	4	31
Statue1 vs Statue3	21	0	2	0	8	0
Tower1 vs Tower2	330	6	86	6	90	80
Tower1 vs Tower3	50	4	15	8	42	56
Chessboard1 vs Chessboard2	50	44	12	3	116	227
Chessboard1 vs Chessboard3	32	20	6	1	183	20
Average Matches	33.4	5.8	9.2	2.1	50.6	30.6

Table 2: Number of matched features for each algorithm across all object pairs.

	Harris + SIFT	Harris + LBP	Blob + SIFT	Blob + LBP	DoG + SIFT	DoG + LBP
kepler1 vs kepler2	0.12	0.05	0.25	0.17	0.14	0.07
kepler2 vs kepler3	0.12	0.06	0.24	0.17	0.14	0.07
Lenna1 vs Lenna2	0.13	0.12	–	–	0.08	0.05
Lenna1 vs Lenna3	–	0.12	–	–	0.07	0.04
Statue1 vs Statue2	0.95	0.77	0.99	–	0.36	0.14
Statue1 vs Statue3	0.31	–	0.35	–	0.17	–
Tower1 vs Tower2	4.63	3.47	2.56	1.42	1.35	0.37
Tower1 vs Tower3	4.53	3.59	2.61	1.43	1.36	0.46
Chessboard1 vs Chessboard2	0.18	0.11	0.56	0.26	0.15	0.07
Chessboard1 vs Chessboard3	0.18	0.11	0.44	0.10	0.16	0.06
Average Time	0.80	0.60	0.62	0.30	0.27	0.10

Table 3: Inference time (seconds) for each algorithm across all object pairs.